

04_ai_engineer

Principal AI Engineer Review

Headline kill. The 85%-vs-45% headline is not a measurement of diagnostic accuracy — it is a tautology dressed as a benchmark. The eval injects anomalies by perturbing aggregates with the *exact* mix/rate/interaction algebra that `Decompose` uses to diagnose (Bible §20.2: ground-truth labels are "the *true* mix/rate/interaction contributions computed analytically by the same decomposition algebra Helios is graded on"), so on the injected data the decomposition is correct *by construction* — it is arithmetically impossible for it to be otherwise. The benchmark therefore grades whether the FSM can run the same equation twice and whether an Opus model can read off the largest term. That is a unit test of `decompose_change`, not evidence that Helios diagnoses real funnels. And even this circular number is a TARGET printed in a results table (§20.9) with no code behind it.

[CRITICAL] The benchmark is circular: ground truth is generated by the system under test

Claim attacked. "root-cause segment accuracy $\geq 85\%$ on the labeled benchmark vs $\leq 45\%$ for the naive baseline" (Bible §20, repeated in CLAUDE.md §4 and AGENT_ARCHITECTURE.md §13 exit gate). The injector "applies one of two perturbation primitives... changes `r_i` only -> ground-truth = **rate-change**" and records "the *true* mix/rate/interaction contributions computed analytically by the same decomposition algebra Helios is graded on" (§20.2). MCP_ARCHITECTURE.md §6.3 confirms `decompose_change` "implements the FOUNDATION identity exactly."

Why it fails. The injector multiplies one cell's rate by `rate_multiplier` while holding `sessions` fixed (§20.2 step 2). Run the identity $\Delta R = \sum \Delta w \cdot r_0 + \sum w_0 \cdot \Delta r + \sum \Delta w \cdot \Delta r$ on that exact mutation and you get, definitionally, `mix=0, rate≠0, interaction=0` — which is precisely what every `single_segment_rate` scenario's `mix_rate_interaction`: {mix: 0.000, rate: -0.013, interaction: 0.000} records (scenarios.yaml S001 et seq.). The "gold target" and the model's answer are computed by the *same function on the same conserved aggregates*. The decomposition MAPE $\leq 10\%$ target (§20.4) cannot fail unless the code has a literal bug; §20.2 even admits "the analytic mix/rate/interaction split is exact." This proves the arithmetic is self-consistent. It proves nothing about whether the system can diagnose an anomaly it did not itself author. A passing benchmark here is compatible with 0% real-world accuracy.

Fundamental. Eval and system share one algebra.

[CRITICAL] Injected anomalies do not resemble real anomalies — only the ones the algebra can see

Claim attacked. "we inject synthetic-but-known anomalies... The pipeline must rediscover what we hid" and the 7-bucket taxonomy (§20.3) presented as the proof the product "diagnoses WHY an e-commerce funnel moved" (thesis, §4).

Why it fails. Every injected scenario lives *inside the dimensional space the tree searches*: a clean rate multiplier on one or two (`channel_group`, `device_category`, `country`, `browser`, ...) cells (scenarios.yaml S001–S032). Real funnel anomalies are the opposite: (1) caused by factors *outside* any GA4 dimension — a deploy at 14:00, a price change, a competitor promo, a payment-processor outage — which the dimensional tree can never name; (2) the product of many overlapping causes with no single dominant cell; (3) instrumentation drift that corrupts the very numbers being decomposed. The benchmark never tests case (1) at all, and the `multi_segment_mixed` bucket is only 6 scenarios of perfectly factored mix+rate (S027–S032). So the eval validates exactly the regime where best-first-by-rate-effect (§19.4) is guaranteed to win and is silent on every regime where it fails. "Rediscover what we hid" is the tell: the system is graded only on findable injections, never on the unfindable causes that dominate real diagnosis.

Fundamental. Real causes live outside the dimension set.

[CRITICAL] "Diagnose WHY" is an overclaim — the engine does arithmetic attribution, not causal inference

Claim attacked. The thesis that Helios "diagnoses WHY an e-commerce funnel moved" (§4) and the Diagnose prompt: "Find the root cause by best-first search" (AGENT_ARCHITECTURE.md §6.4).

Why it fails. `decompose_change` and the hypothesis tree answer *where* the number moved (which cell, mix vs rate), not *why*. Yet the expected diagnoses invent mechanisms wholesale: S021 asserts "a payment SDK regression on iOS WebView," S024 "a shipping-cost estimator failure," S023 "a PDP price-display/strikethrough change." Nothing in the data supports these stories — the injector only multiplied a rate. The mechanism is a confident LLM confabulation layered over a correct number. Grounding (§19.2 G1–G5) guarantees the *columns* are real; it does nothing to make the *causal narrative* real. The benchmark's `expected_diagnosis` strings actually *reward* this confabulation by writing the invented mechanism into the answer key, training/grading the system to assert causes it cannot observe.

Fundamental. Attribution ≠ causation; data lacks the cause.

[CRITICAL] The Critic is a stochastic LLM with no ground truth — "adversarial verification" is not verification

Claim attacked. Trust pillar 3: "adversarial verification... the Critic agent refutes every finding." Critic prompt: "Return PASS only if every refutation fails" (AGENT_ARCHITECTURE.md §6.5); the Critic is Opus, the FSM routes every finding through it (§19.3).

Why it fails. The Critic is another temperature-0-ish Opus sampler with no oracle. It cannot *verify* anything; it can only generate a second opinion about the first opinion. There is no ground truth at runtime — the labels exist only in the eval harness, never in production. "Default to skepticism" is a prompt instruction, not a guarantee; an LLM that hallucinated a plausible iOS-SDK story (Finding) is exactly the LLM most likely to find that story plausible (Critic), because both share training priors and see the same evidence bundle. Worse, the Critic's four refutation axes (confound/sample/seasonality/data-quality) are the same four buckets the eval injects (§20.3) — so the Critic is graded on catching the precise failure modes it was prompted to look for, with the answer pre-seeded into `seasonality_calendar` (see next finding). Who critiques the Critic? Nobody. Majority-of-correlated-samplers is not adversarial robustness.

Fundamental. No oracle; self-grading by correlated models.

[HIGH] The seasonality/data-quality buckets leak their answers through pre-seeded memory

Claim attacked. Seasonality decoys and data-quality scenarios "must NOT flag it (Critic must refute)" (§20.3); the Critic "queries `seasonality_calendar` ... if the observed change is within `expected_mag` of a calendar entry, the finding is DROP'ped" (§22.3).

Why it fails. §22.3 states the `seasonality_calendar` is "seeded once... the dataset spans 2020-11-01→2021-01-31, so Black Friday 2020 and the December peak/January trough are pre-loaded." Now look at the decoy scenarios: S033 post-holiday January trough, S034 Black Friday, S037 Christmas peak, S038 New Year — these are *the exact calendar entries pre-loaded into memory*. The Critic isn't reasoning that a swing is seasonal; it is doing a date-range lookup against a table that was hand-populated with the answer. The `launch_calendar` (§22.3, example "new /sale landing page") similarly pre-loads the mechanism for S006 (/sale landing page) and S023 (PDP change). The benchmark is testing whether a `JOIN` on a date overlaps — trivially passable — and dressing it as "the Critic refutes via `seasonality_calendar`." Take the seeded calendar away (i.e., production with no curated calendar) and the seasonality-decoy accuracy collapses, because there is nothing to look up.

Fundamental. Answers pre-seeded into the lookup table.

[HIGH] "Deterministic FSM with model-driven nodes" is a contradiction that hides where the answer is decided

Claim attacked. "Helios is **not** an autonomous free-roaming agent. It is a **deterministic finite state machine**" (AGENT_ARCHITECTURE.md §1); determinism is sold as Trust Pillar 2.

Why it fails. §11 of the same doc quietly concedes it: "LLM nodes are run at temperature 0 but are **not** byte-deterministic." The FSM determinism covers only *which stage runs next* — PLAN→MONITOR→DECOMPOSE — which is the trivial part. The *diagnosis itself* is whatever the stochastic Opus node decides: which slice to drill (Diagnose's best-first ordering depends on the model choosing what to expand within MAX_BRANCHING=4 / MAX_DEPTH=3 , §9), which causal story to tell, whether the Critic feels skeptical today. The load-bearing decisions are non-deterministic; the bookkeeping around them is deterministic. Selling "deterministic FSM" as a trust pillar conflates reproducible *control flow* with reproducible *answers*, and the latter is the only one that matters for a tool whose output is a causal claim. Two runs on identical frozen data can yield different root causes and different dollar figures — fatal for an "always-on autonomous" product whose runs all see the same static export.

Fundamental. Stochastic nodes decide the answer.

[HIGH] The ≤45% baseline is a strawman engineered to be beaten

Claim attacked. "the naive baseline... ranks by `|delta|` , and declares the single largest-magnitude segment the root cause... scores **~45% top-1**" — "a near-doubling, which is the project's central empirical claim" (§20.5).

Why it fails. The baseline was chosen *because* the benchmark is dominated by buckets it must lose. 12 of 50 scenarios are pure mix-shift (S011–S020) where "largest absolute delta" is designed to be fooled, and the Simpson's-paradox adversarial cases (S014, S018) are constructed specifically so the naive method points at the wrong cell. The "**~45%**" isn't measured — it's asserted ("it gets pure single-segment rate cases right and most mix cases wrong," §20.5), i.e. back-computed from the bucket mix. A serious baseline (per-segment rate-delta ranking, or a chi-square contribution test, or literally "decompose then rank by rate term") would close most of the gap without any of Helios's 7-agent apparatus. Beating a baseline you wrote to lose, on data you injected with your own algebra, is not "the project's central empirical claim" — it is circular twice over.

Fixable. Pick an honest, decomposition-aware baseline.

[HIGH] Cost and latency of 7 agents (3 Opus) + tree + Critic re-query loop cannot credibly fit <5 min / and is unmeasured

Claim attacked. "Time-to-diagnosis <5 min/run" (§5.2) and the results table "Time: 4m12s/run" (§20.9); "This split keeps token cost inside budget" (§19.1).

Why it fails. The results table prints "4m12s" as if measured — it is fabricated; no code exists (CLAUDE.md §10: "No code yet"). Count the real critical path: Orchestrator (Opus) → Monitor (Sonnet, one series per metric over ~22 canonical metrics) → Decompose (Sonnet) → Diagnose (Opus) walking a tree up to depth 3 × branching 4 = up to ~64 nodes, *each* doing build_query→dry_run→run_query→significance_test→decompose_change (≥5 sequential tool round-trips per node) → Critic (Opus) per finding running its own re-queries → up to MAX_REFUTE_ROUNDS=2 *round trips back to Diagnose* (§19.3) → Prescribe → Narrator. Each Opus turn with tool use is multiple seconds; the Diagnose tree alone is hundreds of sequential model+tool round-trips because best-first search is inherently serial (you must see node N's decomposition before choosing N+1). The BYTE_BUDGET=5 GiB is enforced (§4.2) but there is no token budget enforced anywhere, and 3 Opus agents in a refute loop is the most expensive possible topology for a batch job that re-reads the same 3-month static table every run.

High (Fundamental for the latency claim). Serial tree + Opus refute loop.

[HIGH] Faithfulness is checked by an LLM judge — the same failure mode it is meant to catch

Claim attacked. "Faithfulness... does Narrator's prose claim match the SQL evidence... (entailment check)" target ">= 0.95"; "Critic-as-judge entailment pass that flags any sentence not entailed by the evidence bundle" (§20.4, §20.7).

Why it fails. Half the faithfulness check is a deterministic rule (every number maps to a tool-output hash, §20.7 check 1) — fine, but that only catches invented *numbers*, never invented *reasoning*. The other half, entailment, is "Critic-as-judge" — an LLM grading whether an LLM's prose is entailed by evidence. LLM-as-judge for faithfulness is exactly where models are known to be sycophantic and to rubber-stamp fluent, plausible prose. The dangerous output of Helios is a *fluent, plausible, wrong causal story over correct numbers* (see the iOS-SDK / shipping-calculator confabulations in S021/S024) — and that is precisely the output an LLM entailment judge is least able to flag, because the prose *sounds* entailed. The 0.95 target is again unmeasured and self-graded.

Fundamental. LLM judge can't catch fluent-wrong reasoning.

[MEDIUM] BigQuery + vector-store memory is massive over-engineering for a single-tenant batch job on 3 months of frozen data

Claim attacked. "Memory is split across BigQuery tables... and a **vector store** (embeddings of past findings)... `recall_prior` runs a hybrid query — exact filter... plus a vector ANN search... decayed by `exp(-age_days/60)` " (§22.1); `HALF_LIFE=60d` "matching the ~3-month dataset window" (§9, §22.1).

Why it fails. The data is static, historical, single-store, ~3 months. Every "scheduled run" sees the identical frozen export, so "learns across runs" (§22) has nothing to learn from — re-running the same window yields the same findings, and a 60-day half-life on a 92-day dataset means priors decay before a second meaningful cohort even exists. Diagnosis history will hold a handful of rows; ANN vector search over a few dozen findings is theatre — a `WHERE metric=? AND dimension_slice=?` exact match (which §22.1 *also* does) is sufficient and the embedding column is dead weight. The vector store adds an env var (`HELIOS_VECTOR_STORE_URI`), an upsert path, a recall path, and a similarity-decay formula to solve a problem that does not exist at this scale.

Fixable. Drop the vector store; a table suffices.

[MEDIUM] MCP (5 servers, two transports, JSON-RPC error taxonomy) is over-abstraction where in-process function calling would do

Claim attacked. "5 MCP servers" as architecture; the 14-code JSON-RPC error taxonomy (§5), dual transport (stdio + streamable-HTTP, §3), per-server `server_version` manifests, bearer-token auth on warehouse-mcp (§3).

Why it fails. Four of five servers (semantic, stats, experiment, report) run as local stdio subprocesses with "no warehouse credentials" and are "pure functions of their inputs" (§3, §8) — i.e., they are ordinary Python libraries wrapped in JSON-RPC framing for no isolation benefit, since they share the host and trust boundary anyway. The genuine grounding guarantee (the model never holds a raw-SQL handle) is achieved by the *agent tool allow-list* (§10), not by MCP-the-protocol; you get the identical guarantee by registering plain Python tool functions with the Agent SDK. The MCP layer buys serialization overhead, a bespoke error-code registry, subprocess lifecycle management, and a `mcp_servers.yaml` to maintain — for a single-process batch pipeline. The one server that *does* need a boundary (warehouse-mcp, the BigQuery client) could have been the sole tool; instead the doc spins up four credential-free subprocesses to look like a platform.

Fixable. Collapse stdio servers to in-process tools.

[MEDIUM] Allow-list and registry drift contradict the "single source of truth" discipline the trust pillars depend on

Claim attacked. "the §10 doc is the single source of truth" for tools (AGENT_ARCHITECTURE.md §2.1); "registry path = the reconciled canonical `./models/semantic/semantic_models.yml`" (MCP_ARCHITECTURE.md §3, §6.2 binding).

Why it fails. The grounding pillar's whole promise is that names resolve against one governed registry. Yet the docs point `semantic-mcp`'s `registry:` at `models/semantic/semantic_models.yml` (MCP_ARCHITECTURE.md §3 config, CLAUDE.md §5), while the artifact that actually exists and "passed a referential-integrity LINT" is `semantic_layer.yml` — two different filenames for the keystone. AGENT_ARCHITECTURE.md §13's exit gate and §20.9's results table say **34 scenarios**; the shipped `scenarios.yml` header declares **50** across 7 buckets (S001–S050). The Bible §20.3 table sums to 34 with bucket counts (6/6/4/4/4/6/4) that do not match the file's (10/10/6/6/6/6/7). If the team cannot keep the *count of its own benchmark* or the *filename of its own keystone registry* consistent across three docs, the "0 hallucinated columns via the single canonical registry" guarantee is resting on coordination discipline the project has already demonstrably failed.

Fixable. Reconcile names/counts; but it indicts the discipline.